Product Requirements Document (PRD)

BusGuard AI: Predictive Bus Delay & Automated Recovery System

---

Document Control & Meta Information

- **Document Title:** Product Requirements Document — BusGuard AI
- **System Version:** 1.0.0-PROTOTYPE
- **Target Organization:** Rapid Bus Sdn Bhd (Rapid KL / Prasarana Malaysia Berhad)
- **Document Status:** Formal Specification / Hackathon Production Baseline
- **Author/Team:** BusGuard AI Engineering Team
- **Target Runtime Environment:** Python 3.10+ / Streamlit / OR-Tools / Scikit-Learn

---

1. Executive Summary & Product Vision

1.1 Problem Context

Rapid KL bus networks in the Klang Valley operate under dense urban road conditions where traffic congestion, unexpected bottlenecks, and weather anomalies degrade scheduled timetables. Current operations rely heavily on reactive dispatch protocols: human controllers observe severe delays only after trip bunching or stranded commuter queues materialize.

1.2 Product Vision

BusGuard AI transitions public transit fleet management from **reactive damage control** to **proactive, closed-loop mitigation**. By leveraging telemetry, historical travel times, and schedule baselines, BusGuard AI detects impending delay cascades before they propagate, automatically evaluates multi-variable operational interventions (such as standby bus injection or headway smoothing), determines autonomous execution safety vs. operator sign-off, and verifies post-intervention recovery outcomes within a discrete event simulation sandbox.

1.3 Core Operating Tenets

1. **Never Hallucinate Fleet Realities:** The system strictly separates verified telemetry, statistical machine learning inferences, and simulated sandbox outcomes.

2. **Safety-First Intervention:** Prediction confidence alone does not authorize automated dispatch. Execution eligibility requires satisfaction of feasibility, data integrity, and strict risk thresholds.
3. **Transparent Explainability:** Every alert, priority score, optimization trade-off, and execution route must expose clear, traceable reasoning to human dispatchers.

---

2. Product Scope & Operational Boundaries

```
None
+-------------------------------------------------------------------------------
------+
|                                    BUSGUARD AI SCOPE
|
+-----------------------------------+-----------------------------------------
------+
| IN SCOPE (Prototype & Sandbox)    | OUT OF SCOPE (Strict Prohibitions)
|
+-----------------------------------+-----------------------------------------
------+
| - GPS Telemetry Ingestion & EDA   | - Direct CAN-bus / On-board Hardware
Control     |
| - Supervised ML Delay Forecasting | - Live Dispatch to Real Rapid KL Road
Fleets     |
| - Feasibility & Constraint Engine | - Driver Telematics Mobile Apps (Native
iOS/And) |
| - Heuristic/MILP Action Optimizer | - Autonomous Payroll / Shift Overtime
Accounting |
| - Streamlit Recovery Workbench    | - Commuter Fare Collection / Touch 'n Go
Sync    |
| - Simulation-Based Impact Tracking| - Fabricated Accuracy / Synthetic Masking
|
+-----------------------------------+-----------------------------------------
------+
```

---

3. User Personas & System Stakeholders

Persona A: Rapid KL Chief Route Controller (Ahmad)

- **Role:** Central Operations Command Centre (OCC) Senior Controller.

- **Goals:** Eliminate 45+ minute passenger stranding during peak commute hours (07:00–09:30, 17:00–20:00).
- **Pain Points:** Overwhelmed by raw unstructured GPS pings; unable to cross-reference depot reserve availability during rapid traffic deterioration.
- **Workflow in BusGuard AI:** Reviews high-risk queued trips; approves or rejects amber-tier dynamic interventions with single-click operational insight.

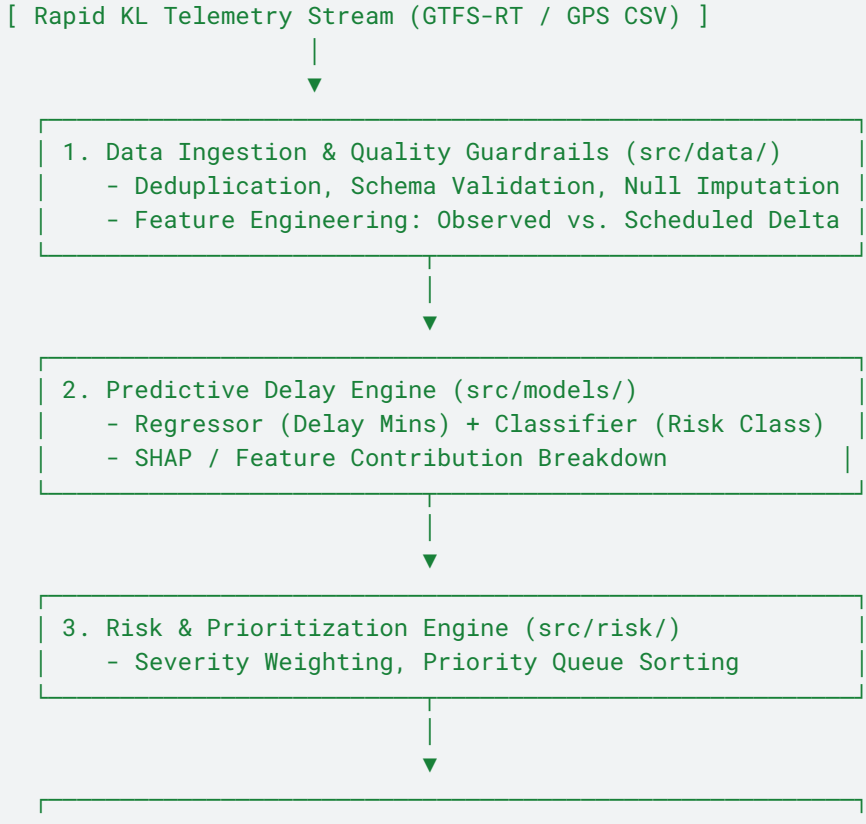
Persona B: Fleet Resource Dispatcher (Norah)

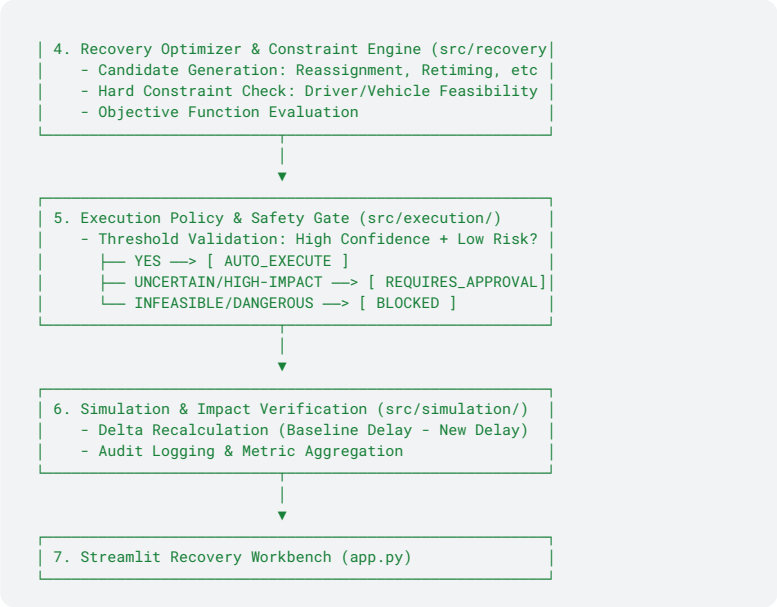
- **Role:** Subang / Cheras Depot Bus Allocator.
- **Goals:** Maximize utilization of reserve and layover standby buses without depleting statutory off-peak maintenance buffers.
- **Pain Points:** Lack of forward notice on where stranded passengers will bottleneck next.
- **Workflow in BusGuard AI:** Observes AI-recommended standby injections; tracks system-allocated assets to prevent over-allocation.

---

4. End-to-End System Architecture & Information Architecture

None





---

5. Functional Requirements (FR-01 to FR-42)

The system is strictly designed around the **Four Operational Pillars**: **DETECT**, **CATEGORISE**, **PRIORITISE**, and **EXPLAIN**.

Feature 1: Predictive Delay Intelligence & Risk Prioritisation

ID	Title	Requirement Description	Pillar	Acceptance Criteria
FR-01	Telemetry Ingestion	Ingest and validate tabular/streaming bus movement data (Bus ID, Route ID, Latitude, Longitude, Timestamp,	DETECT	Ingests CSV/JSON datasets with zero unhandled null schema aborts.

ID	Title	Requirement Description	Pillar	Acceptance Criteria
		Scheduled Arrival, Actual Arrival).		
FR-02	Target Derivation	Compute ground-truth delay target $Y_{\text{delay}} = T_{\text{observed}} - T_{\text{scheduled}}$ in minutes.	DETECT	Rejects records where timestamps are missing or negative without logging.
FR-03	Model Inference	Feed temporal, spatial, and headway features into trained ML estimators (Random Forest / XGBoost) to forecast trip delay.	DETECT	Returns continuous predicted delay $\hat{d}$ for all eligible active trips.
FR-04	Horizon Detection	Flag upcoming trips projected to exceed critical delay thresholds prior to reaching scheduled checkpoints.	DETECT	Identifies trip risks at least 15–30 minutes ahead in simulated time.
FR-05	Uncertainty / Probabilities	Calculate probability $P(\text{delay} \ge 15\text{ min})$ alongside the continuous point prediction.	DETECT	Exposes classification confidence as a bounded float $[0.0, 1.0]$.
FR-06	Risk Categorisation	Bin trip risk into four distinct tiers based on delay duration and probability thresholds: `LOW`, `MEDIUM`, `HIGH`, `CRITICAL`.	CATEGORISE	Thresholds follow §7.2 specifications precisely.
FR-07	Root Cause Identification	Identify primary delay factor (e.g., Arterial Gridlock, Upstream Bunching, Terminal Turnaround Deficit) from available feature indicators.	CATEGORISE	Flags root cause only when feature threshold indicators confirm correlation.
FR-08	Dynamic Prioritization	Compute an operational Priority Score for each trip using the formal formula in §7.3.	PRIORITISE	Ranks trips in descending priority order on the dispatcher queue.
FR-09	Dynamic Reranking	Dynamically update priority queue upon telemetry refresh or simulation event.	PRIORITISE	Recomputes priority order in < 1.5 seconds upon dataset push.

ID	Title	Requirement Description	Pillar	Acceptance Criteria
FR-10	Status Display	Render trip ID, route code, predicted delay, risk tier, priority rank, and stop identifiers on the UI.	EXPLAIN	All metrics visible in single-pane master view.
FR-11	Local Explainability	Expose feature contributions (e.g., top 3 SHAP or feature weight contributors) behind each delay prediction.	EXPLAIN	Displays human-readable rationale (e.g., *"+18 min due to Federal Highway corridor congestion"*).
FR-12	Data Reliability Warning	Explicitly mark predictions with low sensor confidence or missing telemetry as `DATA_UNRELIABLE`.	DETECT / EXPLAIN	Prevents high-risk auto-execution when input quality is degraded.

Feature 2: AI Recovery Optimisation & Action Selection

ID	Title	Requirement Description	Pillar	Acceptance Criteria
FR-13	Action Generation	Instantiate candidate recovery interventions for trips exceeding `MEDIUM` risk classification.	DETECT	Evaluates candidate set for every qualifying trip.
FR-14	Strategy Repertoire	Support evaluation across 4 discrete intervention archetypes: (1) Route/Departure Retiming, (2) Standby-Bus Reassignment, (3) Headway/Departure Smoothing, and (4) Baseline (No-Action).	DETECT	System evaluates all 4 archetypes before final action selection.
FR-15	Feasibility Filtering	Categorize each candidate as `FEASIBLE`, `INFEASIBLE`, or `UNEVALUABLE` based on constraint checks.	CATEGORISE	Infeasible actions are never proposed for execution.
FR-16	Hard Constraint Checks	Verify driver shift limits, standby vehicle physical availability, depot travel distance, and downstream trip assignments.	CATEGORISE	Disallows standby assignment if deadhead time exceeds delayed bus arrival delta.

ID	Title	Requirement Description	Pillar	Acceptance Criteria
FR-17	Delay Offset Projection	Compute projected post-action delay $\hat{d}_{\text{post}}$ using verified simulation rules.	DETECT	Post-action delay calculation strictly adheres to §7.4 formulas.
FR-18	Action Trade-Off Metric	Calculate marginal benefit-to-cost ratio considering delay saved, fuel/driver operating cost, and passenger disruptions.	PRIORITISE	Ranks feasible candidate interventions according to objective score.
FR-19	Optimal Action Selection	Select highest-scoring feasible candidate via defined objective function $J(\text{action})$.	PRIORITISE	Algorithm deterministically chooses the optimal feasible strategy.
FR-20	Baseline Preservation	If no candidate yields improvement greater than configured threshold $\epsilon_{\text{threshold}}$, select `NO_ACTION` baseline.	PRIORITISE	Prevents unnecessary operational disruption for negligible gains.
FR-21	Decision Rationale	Provide concise, plain-language operational justification for the chosen action and why alternatives were rejected.	EXPLAIN	Workbench explains: *"Standby Bus R1 selected: saves 32 mins. Route modification rejected: exceeds maximum route deviation bounds."*
FR-22	Assumption Disclosure	Clearly list all assumed parameters (e.g., assumed deadhead speed, depot turnaround) when evaluating unmeasured actions.	EXPLAIN	Labels simulated parameters with `[ASSUMPTION]` tags on the UI.

Feature 3: Confidence-Based Automatic Execution & Human Approval

ID	Title	Requirement Description	Pillar	Acceptance Criteria
FR-23	Safety Gate Evaluation	Pass selected action through verification policy: model confidence, data freshness, operational severity, and safety boundary limits.	DETECT	Evaluation occurs prior to any simulation execution command.
FR-24	Execution Decision State	Classify execution mode into exactly one of three states: `AUTO_EXECUTE`, `REQUIRES_APPROVAL`, or `BLOCKED`.	CATEGORISE	State assigned according to decision matrix in §7.5.
FR-25	Autonomous Simulation	If classified as `AUTO_EXECUTE`, immediately apply the intervention to the simulation state without human intervention.	PRIORITISE / EXECUTE	State changes instantly reflected with system signature `DISPATCH_DAEMON_AI`.
FR-26	Human Approval Gate	If classified as `REQUIRES_APPROVAL`, queue intervention in pending approval tray; require human controller input.	PRIORITISE	Action is held indefinitely until controller intervenes.
FR-27	Pre-Execution Briefing	Present full decision context on the approval panel: trip details, proposed action, predicted delay reduction, risk flags, and operational costs.	EXPLAIN	Complete briefing visible before action buttons are active.
FR-28	Controller Interaction	Provide simple interactive controls: `[APPROVE ACTION]` and `[REJECT / KEEP BASELINE]`.	EXECUTE	User button clicks instantly mutate trip intervention state.
FR-29	Decision Enactment	Approved actions execute into simulation sandbox; rejected actions revert to baseline and record controller sign-off.	EXECUTE	Rejected actions are logged and never applied to simulation.

ID	Title	Requirement Description	Pillar	Acceptance Criteria
FR-30	Audit Log Capture	Generate append-only audit record detailing timestamp, Trip ID, model confidence, decision state, action parameters, and controller ID.	EXPLAIN	Every transaction recorded in JSON/CSV audit ledger.
FR-31	Fallback Guarantee	If an action is blocked or fails during execution, immediately restore original baseline timetable.	PRIORITISE	Simulation state remains consistent and deadlock-free.
FR-32	Idempotency Protection	Guard against duplicate execution for the same trip and time window.	EXECUTE	Duplicate execution requests on active trips return HTTP/Application `409 CONFLICT` state.

Feature 4: Recovery Impact Calculation & Verification

ID	Title	Requirement Description	Pillar	Acceptance Criteria
FR-33	Baseline Snapshot	Record immutable baseline total predicted delay `$\sum d_{\text{baseline}}$` prior to intervention execution.	DETECT	Baseline persisted in transaction state.
FR-34	Post-Action Recalculation	Recompute trip progression and delay profiles within the simulation following intervention deployment.	DETECT	Post-action state updated within < 1.0 second of execution.
FR-35	Outcome Classification	Classify post-action result: `SUCCESSFUL`, `PARTIALLY_EFFECTIVE`, `INEFFECTIVE`, or `INFEASIBLE`.	CATEGORISE	Categorization matches mathematical bands in §7.6.
FR-36	Absolute Delay Delta	Compute `$\Delta_{\text{delay}} = d_{\text{baseline}} -`	PRIORITISE	Arithmetic checked against baseline snapshot.

ID	Title	Requirement Description	Pillar	Acceptance Criteria
		d_{post} in integer minutes.		
FR-37	Percentage Improvement	Compute $\text{Improvement} = \left(\frac{\Delta_{\text{delay}}}{d_{\text{baseline}}}\right) \times 100\%$ with zero-division handling.	PRIORITISE	If $d_{\text{baseline}} \le 0$, percentage improvement is explicitly reported as 0.0%.
FR-38	Trip Impact Delta Matrix	Break down system-wide trip effects: list count and IDs of trips Improved, Unchanged, or Negatively Impacted.	PRIORITISE	Reports downstream ripple effects on connecting lines.
FR-39	Cost Impact Ledger	Compute incremental operational expenditure Δ_{cost} (fuel + driver wages) if cost data exists; else display `COST_DATA_SIMULATED`.	PRIORITISE	Never asserts real-dollar savings without verified unit-cost schema.
FR-40	Objective Verification	Validate whether the executed intervention achieved its projected objective within a tolerance band ($\pm 15\%$).	DETECT	Flags discrepancies between projected vs. simulated outcomes.
FR-41	Post-Mortem Rationale	Produce an automated debrief explaining why an intervention succeeded or underperformed.	EXPLAIN	Explains operational variances in human-readable terms.
FR-42	Historical Impact Ledger	Retain all historical intervention runs in an accessible verification session ledger for controller post-shift reviews.	EXPLAIN	Historical ledger fully exportable to CSV/JSON format.

6. Non-Functional Requirements (NFR-01 to NFR-30)

None

```
+-------------------------------------------------------------------------------------------------+
|                                NON-FUNCTIONAL REQUIREMENTS MATRIX                               |
+--------+------------------------+--------------------------------+-----------------------------+
| ID     | Requirement Category   | Performance / Quality Target   | Metric / Verification       |
+--------+------------------------+--------------------------------+-----------------------------+
| NFR-01 | Model Evaluation       | Regression: MAE <= 4.5 mins;   | 5-Fold Temporal Split CV    |
|        | Performance            | Classifier: F1-Score >= 0.78   | on historical Rapid KL log  |
| NFR-02 | Prediction Latency     | <= 3.0 seconds per batch       | Measured under standard     |
|        |                        | of 100 active trips            | Python 3.10 runtime         |
| NFR-03 | Telemetry Validation   | 100% schema check & sanitise   | Reject null IDs & GPS inf   |
| NFR-04 | Rationale Transparency | 100% displayed decisions must  | Rationale text present      |
|        |                        | provide structured rationale   | on all UI action cards      |
| NFR-05 | Determinism            | Identical inputs produce       | Test with fixed random      |
|        |                        | identical optimization states  | seed across optimizer       |
| NFR-06 | Modular Architecture   | Decoupled components in `src/` | Zero circular dependencies  |
| NFR-07 | Graceful Error Handling| UI uptime 100% during invalid  | Try/except fallbacks with   |
|        |                        | input injection                | standard error toasts       |
| NFR-08 | Model Reproducibility  | Full artifact versioning       | Track model hash, hyper-    |
|        |                        | (Joblib / MLflow registry)     | parameters, and split date  |
| NFR-09 | Constraint Integrity   | 0% hard constraint violations  | Zero infeasible dispatches  |
| NFR-10 | Optimizer Latency      | <= 5.0 seconds per trip        | Benchmarked on OR-Tools     |
| NFR-11 | UI Decision Rendering  | <= 500 ms rendering update     | Streamlit session delta     |
```

```
| NFR-12 | Financial Integrity    | No fabricated RM claims       | Simulated
costs tagged    |
| NFR-13 | Symmetrical Comparison | Baseline and action use equal | Same random
seed & traffic|
|        |                        | simulation noise parameters   | density matrix
|
| NFR-14 | Module Extensibility   | Plugin interface for actions  | BaseStrategy
ABC class    |
| NFR-15 | Safe Optimization Fail | Fallback to `NO_ACTION` baseline| Assert
system integrity   |
| NFR-16 | Safe Execution Policy  | 0 unauthorized auto-dispatches| Check against
rule matrix |
| NFR-17 | Threshold Validation   | Dynamic config via YAML/JSON  | Read from
`config.yaml`   |
| NFR-18 | Controller Sovereignity| User action required on amber | UI buttons
mandatory      |
| NFR-19 | Audit Immutability     | Append-only event logger      | File write
permissions 644|
| NFR-20 | Idempotency Key        | Unique UUID per trip-incident | Duplicate keys
rejected   |
| NFR-21 | Simulation Rollback    | 100% rollback on sim failure  | Deepcopy state
snapshot   |
| NFR-22 | Prototype Security     | Input sanitize against inject | Regex
validation on text  |
| NFR-23 | Environment Sandbox    | Physical vehicle isolation    | Zero external
network I/O |
| NFR-24 | Formula Precision      | IEEE 754 Floating Precision   |
Double-precision floats   |
| NFR-25 | Arithmetic Safety      | Zero division exceptions      | Guard clauses
on % deltas |
| NFR-26 | Complete Traceability  | Foreign key link across logs  | IncidentUUID
in all tables|
| NFR-27 | UI Metric Latency      | <= 3.0s impact recalculation  | End-to-end
benchmarked    |
| NFR-28 | Truthful Labeling      | Explicit `[SIMULATED]` badges | UI CSS badge
compliance   |
| NFR-29 | Unit Uniformity        | Metric system (km, km/h, min)  | Strict unit
standards     |
| NFR-30 | Incomplete Sim Fail    | Flag unverified outcomes      | State =
`UNVERIFIED`      |
+--------+------------------------+-------------------------------+---------------
------------+
```

---

7. Mathematical Formulations & Decision Logic

7.1 Delay Prediction & Ground-Truth Formulation

Given trip i at scheduled checkpoint stop k: $$T_{\text{observed}}(i, k) = \text{Actual arrival timestamp}$$ $$T_{\text{scheduled}}(i, k) = \text{Timetable baseline arrival timestamp}$$ $$\text{Delay}_{\text{actual}}(i, k) = \max\left(0, \; \frac{T_{\text{observed}}(i, k) - T_{\text{scheduled}}(i, k)}{60}\right) \quad \text{[minutes]}$$

The predictive model estimates continuous delay $\hat{d}_i$ and delay probability P_i: $$\hat{d}_i = f_{\theta}\left(\mathbf{x}_i\right)$$ $$P_i = P\left(\hat{d}_i \ge \tau_{\text{threshold}} \mid \mathbf{x}_i\right) = \sigma\left(g_{\phi}\left(\mathbf{x}_i\right)\right)$$ Where $\mathbf{x}_i$ is the engineered feature vector containing current spatial coordinates, weather index, time-of-day sin/cos encoding, headway to leading vehicle, and segment historical speed.

7.2 Risk Categorization Thresholds

Trip risk $R_i \in \{\text{LOW}, \text{MEDIUM}, \text{HIGH}, \text{CRITICAL}\}$ is partitioned via:

R_i = \begin{cases} \text{CRITICAL}, & \text{if } \hat{d}_i \ge 35\text{ mins} \quad \text{OR} \quad (\hat{d}_i \ge 25\text{ mins} \;\wedge\; P_i \ge 0.85) \\ \text{HIGH}, & \text{if } 20\text{ mins} \le \hat{d}_i < 35\text{ mins} \quad \text{OR} \quad (\hat{d}_i \ge 15\text{ mins} \;\wedge\; P_i \ge 0.70) \\ \text{MEDIUM}, & \text{if } 10\text{ mins} \le \hat{d}_i < 20\text{ mins} \\ \text{LOW}, & \text{if } \hat{d}_i < 10\text{ mins} \end{cases}

7.3 Operational Priority Scoring Formula

To rank multiple concurrent delayed trips for optimization and human dispatcher review:

$$\text{PriorityScore}(i) = w_1 \cdot \left(\frac{\hat{d}_i}{\hat{d}_{\max}}\right) + w_2 \cdot P_i + w_3 \cdot U_i + w_4 \cdot \left(\frac{C_i}{C_{\max}}\right)$$

Where:

- $\hat{d}_i$: Predicted delay of trip i.
- $\hat{d}_{\max}$: Normalization ceiling (default: 60 minutes).
- P_i: Probability of delay propagation $[0, 1]$.
- U_i: Urgency factor based on time until scheduled arrival T_{eta}: $$U_i = \exp\left(-\frac{\max(0, T_{\text{eta}})}{30}\right)$$
- C_i: Commuter load / stop passenger density index (if available; otherwise defaulted to 1.0).
- Weights configuration: $w_1 = 0.40, \; w_2 = 0.25, \; w_3 = 0.20, \; w_4 = 0.15$ (where $\sum w = 1.0$).

7.4 Optimization Objective & Constraint Bounds

Let $\mathcal{A}_i = \{a_0, a_1, a_2, a_3\}$ be the candidate recovery action set for trip i:

- a_0: Baseline (`NO_ACTION`)
- a_1: Standby-bus reassignment
- a_2: Route / departure adjustment
- a_3: Adaptive headway smoothing

The recovery engine selects action $a^* \in \mathcal{A}_i$ that maximizes net operational utility $J(a)$:

$$J(a) = \alpha \cdot \Delta d(a) - \beta \cdot \text{Cost}_{\text{normalized}}(a) - \gamma \cdot \text{DisruptionRisk}(a)$$

Subject to Hard Constraints:

1. **Physical Deadhead Arrival:** $$T_{\text{standby_reach}}(a) \le T_{\text{intercept_target}} - \delta_{\text{buffer}}$$
2. **Driver Availability:** $$\text{ShiftElapsed}(\text{Driver}_j) + \text{Duration}(a) \le \text{ShiftLimit}_{\max} \quad (8.0\text{ hours})$$
3. **Standby Depot Reservation:** $$\text{DepotReserveCount}(\text{Depot}_k) - 1 \ge \text{DepotMinimumBuffer} \quad (1\text{ vehicle})$$

4. **Minimum Recovery Viability:** $$\Delta d(a) \ge 7.0\text{ minutes} \quad (\text{otherwise action rejected for } a_0)$$

Where:

- $\Delta d(a) = \hat{d}_{\text{baseline}} - \hat{d}_{\text{post}}(a)$
- $\alpha = 0.60, \; \beta = 0.25, \; \gamma = 0.15$

7.5 Execution Decision Logic Matrix

The Execution Safety Gate evaluates action a^* against the policy matrix:

None

```
+-------------------------------------------------------------------------------------------------+
|                                  EXECUTION DECISION LOGIC MATRIX                                |
+--------------------+-----------------+----------------------+--------------------+---------------+
| Model Confidence   | Action Risk     | Operational Impact   | Constraint Status  | Decision State |
| (Conf >= 0.85)     | (Disruption)    | (Delay Delta)        |                    |                |
+--------------------+-----------------+----------------------+--------------------+---------------+
| TRUE               | LOW             | Delta >= 10 mins     | ALL SATISFIED      | AUTO_EXECUTE   |
| TRUE               | MEDIUM / HIGH   | Any Delta            | ALL SATISFIED      | REQUIRES_APPV  |
| FALSE (< 0.85)     | ANY             | Any Delta            | ALL SATISFIED      | REQUIRES_APPV  |
| ANY                | ANY             | Delta < 7 mins       | ALL SATISFIED      | BLOCKED (Keep) |
| ANY                | ANY             | ANY                  | VIOLATED           | BLOCKED (Fail) |
+--------------------+-----------------+----------------------+--------------------+---------------+
```

7.6 Impact Verification & Recovery Categorization

Following simulated execution, recovery effectiveness is classified according to achieved percentage delay reduction:

$$\text{Improvement} = \begin{cases} \left(\frac{d_{\text{baseline}} - d_{\text{post}}}{d_{\text{baseline}}}\right) \times 100\%, & \text{if } d_{\text{baseline}} > 0 \\ 0.0\%, & \text{if } d_{\text{baseline}} \le 0 \end{cases}$$

$$\text{RecoveryStatus} = \begin{cases} \text{SUCCESSFUL}, & \text{if } \text{Improvement} \ge 40.0\% \\ \text{PARTIALLY_EFFECTIVE}, & \text{if } 15.0\% \le \text{Improvement} < 40.0\% \\ \text{INEFFECTIVE}, & \text{if } 0.0\% \le \text{Improvement} < 15.0\% \\ \text{DEGRADED}, & \text{if } \text{Improvement} < 0.0\% \quad (d_{\text{post}} > d_{\text{baseline}}) \end{cases}$$

---

8. Data Schema & Ingestion Specifications

8.1 Primary Dataset Schema (`rapidkl_telemetry.csv`)

```
None
+-------------------+--------------+----------+-----------------------------------
----------------+
| Field Name        | Data Type    | Nullable | Operational Description
|
+-------------------+--------------+----------+-----------------------------------
----------------+
| trip_id           | String (UUID)| False    | Unique identifier for scheduled
trip run           |
| route_id          | String       | False    | Rapid KL route identifier (e.g.,
"750", "BET3")   |
| bus_id            | String       | False    | Physical vehicle registration
(e.g., "WXD8821")   |
| stop_id           | String       | False    | Target Rapid KL bus stop
checkpoint code          |
| scheduled_time    | ISO 8601     | False    | Timetable scheduled arrival
timestamp            |
| observed_time     | ISO 8601     | True     | Actual telemetric checkpoint
arrival timestamp    |
| latitude          | Float64      | False    | WGS84 GPS latitude coordinate
|
| longitude         | Float64      | False    | WGS84 GPS longitude coordinate
|
| speed_kmh         | Float32      | False    | Real-time instantaneous speed
|
```

```
| congestion_index  | Float32      | True     | External road congestion score
[0.0 - 1.0]        |
| passenger_load    | Integer      | True     | On-board passenger estimate via
ticketing delta   |
+-------------------+--------------+----------+---------------------------------
----------------+
```

8.2 Standby Asset Schema (`standby_fleet_registry.json`)

```JSON
{
  "standby_units": [
    {
      "standby_id": "STBY-SUBANG-01",
      "depot_id": "DEPOT-SUBANG",
      "vehicle_id": "WA1190B",
      "capacity": 65,
      "status": "IDLE_AVAILABLE",
      "driver_id": "DRV-8841",
      "driver_shift_remaining_mins": 260,
      "current_lat": 3.0567,
      "current_lon": 101.5851,
      "dispatch_readiness_mins": 5
    }
  ]
}
```

---

9. User Interface & Recovery Workbench Specification

The UI is built as a single-page, operator-grade **Recovery Workbench** in Streamlit, structured into four horizontal operational zones:

```None
+-------------------------------------------------------------------------------
----------------+
|  BUSGUARD AI - RAPID KL OPERATIONS RECOVERY WORKBENCH              [LIVE
SIMULATOR: CONNECTED] |
+-------------------------------------------------------------------------------
----------------+
```

```
| ZONE 1: OPERATIONAL TELEMETRY & SYSTEM HEALTH
|
| [ Active Trips: 142 ]  [ At-Risk: 18 ]  [ Standby Fleet Avail: 4 ]  [
Auto-Executions: 7 ]     |
+-------------------------------------------------------------------------------
----------------+
| ZONE 2: DISPATCH QUEUE: PRIORITIZED AT-RISK TRIPS
|
| Trip ID  | Route | Scheduled | Pred. Delay | Risk Level | Priority | Likely
Cause  | Action     |
| TR-9821  | 750   | 18:15     | +38.5 mins  | CRITICAL   | 0.94     | Federal
Bottle| [SELECT]   |
| TR-4102  | 772   | 18:22     | +24.0 mins  | HIGH       | 0.76     | Upstream
Clump| [SELECT]   |
+-------------------------------------------------------------------------------
----------------+
| ZONE 3: INCIDENT DEEP DIVE & AI ACTION RESOLUTION (Selected: TR-9821)
|
| ┌──────────────────────────────────────┐
┌──────────────────────────────────────────────────┐ |
| | Explainability & Feature Impact      | | Recommended Strategy: Standby Bus
Reassignment   | |
| | - Historical Segment Congestion: +55% | | Target Bus: STBY-SUBANG-01 (Depot
Subang)         | |
| | - Downstream Rain Anomaly: +25%      | | Feasibility: VALIDATED (Deadhead: 8
mins)        | |
| | - Speed Deficit (12 km/h): +20%      | | Projected Delay Reduction: -28.0
mins (-72%)     | |
| └──────────────────────────────────────┘
└──────────────────────────────────────────────────┘ |
|
|
| [ EXECUTION SAFETY STATUS: REQUIRES_APPROVAL ]
|
| Rationale: High passenger impact on primary trunk corridor requires controller
sign-off.       |
|
|
| [ APPROVE AND DISPATCH TO SIMULATION ]      [ REJECT INTERVENTION / RETAIN
BASELINE ]           |
+-------------------------------------------------------------------------------
----------------+
| ZONE 4: CLOSED-LOOP SIMULATION IMPACT VERIFICATION
|
| Metric                   | Pre-Intervention Baseline | Post-Intervention
Simulated | Variance   |
| Total Route 750 Delay    | 52.0 mins                 | 14.0 mins
| -38.0 mins |
```

```
| Est. Stranded Passengers | 84 commuters              | 0 commuters
| -100%      |
| Operating Delta Cost     | RM 0.00                   | RM 28.50 [SIMULATED]
| +RM 28.50  |
| Final Recovery Status    | SUCCESSFUL (73.1% Recovery Efficiency)
|
+-------------------------------------------------------------------------------
----------------+
```

---

10. Technical Stack & Repository Architecture

10.1 Technology Stack & Dependency Constraints

None

```
+-------------------+--------------------+---------------------------------------
----------------+
| Layer             | Library / Tool     | Version Constraint & Purpose
|
+-------------------+--------------------+---------------------------------------
----------------+
| Core Language     | Python             | >= 3.10, < 3.12
|
| Data Processing   | Pandas / NumPy     | pandas>=2.0.0, numpy>=1.24.0
|
| Machine Learning  | Scikit-Learn       | scikit-learn>=1.3.0 (RandomForest /
GradientBoosting)  |
| Advanced ML (Opt) | XGBoost            | xgboost>=1.7.0 (Gradient boosted
decision trees)       |
| Explainability    | SHAP               | shap>=0.42.0 (TreeExplainer feature
attribution)       |
| Optimization      | Google OR-Tools    | ortools>=9.7.0 (Constraint Programming
/ MILP)         |
| UI Framework      | Streamlit          | streamlit>=1.28.0 (Reactive web
front-end)            |
| Testing Suite     | Pytest             | pytest>=7.4.0 (Unit, integration, and
property tests)  |
+-------------------+--------------------+---------------------------------------
----------------+
```

10.2 File Organization & Directory Structure

None

```
busguard-ai/
├── app.py                          # Streamlit Recovery Workbench entry-point
├── config/
│   ├── thresholds.yaml             # Risk tiers, safety limits, model weights
│   └── routes_metadata.json        # Rapid KL route coordinates and stop metadata
├── data/
│   ├── raw/                        # Telemetry dumps (CSV/JSON)
│   ├── fixtures/                   # Deterministic test cases (Golden fixtures)
│   └── synthetic/                  # Clearly labelled fallback simulation datasets
├── models/
│   └── delay_rf_v1.joblib          # Serialized model artifact
├── src/
│   ├── __init__.py
│   ├── data/
│   │   ├── loader.py               # Ingestion, timestamp parsing, schema
validator
│   │   └── preprocessor.py         # Feature engineering, cyclical time encoding
│   ├── models/
│   │   ├── train.py                # Temporal split train script
│   │   ├── predict.py              # Model inference wrapper
│   │   └── explain.py              # Feature contribution / SHAP calculators
│   ├── risk/
│   │   ├── categorizer.py          # Binning: Low, Medium, High, Critical
│   │   └── ranker.py               # Priority score sorting implementation
│   ├── recovery/
│   │   ├── generator.py            # Evaluates candidate intervention archetypes
│   │   ├── constraints.py          # Hard constraint validators (driver, standby)
│   │   └── optimizer.py            # Objective function maximizing utility
│   ├── execution/
│   │   ├── policy.py               # Auto vs Approval vs Block decision gate
│   │   └── handler.py              # State mutation executor
│   ├── simulation/
│   │   └── engine.py               # Discrete post-action state delta engine
│   ├── verification/
│   │   └── metrics.py              # Delta delay, percentage improvement, cost
calc
│   └── audit/
│       └── logger.py               # Immutable append-only audit ledger
├── tests/
│   ├── test_data_pipeline.py       # Schema and missing value handling tests
│   ├── test_models.py              # Predictor MAE benchmark & determinism tests
│   ├── test_optimizer.py           # Feasibility check & baseline selection tests
│   ├── test_execution_safety.py    # Safety gate auto-execute vs approval tests
│   └── test_verification.py        # Math precision & zero-division tests
├── requirements.txt                # Pinned dependencies
```

```
├── README.md                        # Operational setup & reproduction guide
└── PRD.md                           # Formal specification document
```

---

11. Testing & Quality Assurance Plan

None

```
+-------------------------------------------------------------------------------
----------------+
|                                  TESTING SUITE SPECIFICATIONS
|
+---------------------------+-----------------------------------+----------------
----------------+
| Test File                 | Coverage Scope                    | Pass Criteria
|
+---------------------------+-----------------------------------+----------------
----------------+
| test_data_pipeline.py     | Null injection, corrupt GPS pings,| Graceful
rejection; zero        |
|                           | invalid datetime formats          | unhandled
exceptions.           |
| test_models.py            | Temporal data leakage prevention, | Model MAE <= 4.5
mins;          |
|                           | output probability range [0, 1]   | identical
predictions on seed.  |
| test_optimizer.py         | Zero standby asset condition,     | Reverts cleanly
to NO_ACTION;   |
|                           | invalid deadhead distances        | zero hard
constraint breaches.  |
| test_execution_safety.py  | Low confidence threshold triggers,| 100% routed to
APPROVAL or      |
|                           | high disruption impact actions    | BLOCKED; zero
unauthorized runs.|
| test_verification.py      | Divide-by-zero on 0-delay trips;  | Safe 0.0%
report; mathematical  |
|                           | precision on delta calculations   | integrity
verified.             |
| test_end_to_end.py        | Full loop: Ingest -> Predict ->   | End-to-end run
passes           |
|                           | Optimize -> Approve -> Verify     | in <= 4.0
seconds execution.    |
+---------------------------+-----------------------------------+----------------
----------------+
```

12. Implementation Roadmap & Milestones

```
None
+-------------------------------------------------------------------------------
------+
| MILESTONE 1: DATA & PREDICTION FOUNDATION (Phase 1)
|
| - Implement src/data/ loader with strict schema integrity validation.
|
| - Train baseline Random Forest / XGBoost model using temporal train/test split.
|
| - Validate MAE benchmark and save serialized pipeline model artifact.
|
+-------------------------------------------------------------------------------
------+
                                          |
                                          ▼
+-------------------------------------------------------------------------------
------+
| MILESTONE 2: RISK, OPTIMIZATION & RECOVERY ENGINE (Phase 2)
|
| - Implement risk categorization logic and dynamic priority scoring engine.
|
| - Develop candidate action generator: Reassignment, Retiming, Smoothing,
Baseline.    |
| - Implement hard constraint solver checking standby bus and driver parameters.
|
+-------------------------------------------------------------------------------
------+
                                          |
                                          ▼
+-------------------------------------------------------------------------------
------+
| MILESTONE 3: SAFETY GATE, SIMULATOR & WORKBENCH (Phase 3)
|
| - Implement execution safety matrix (AUTO_EXECUTE vs REQUIRES_APPROVAL vs
BLOCKED).   |
| - Build discrete-event simulation engine to compute post-action delay
reductions.     |
| - Construct Streamlit Recovery Workbench featuring all 4 operational zones.
|
+-------------------------------------------------------------------------------
------+
                                          |
```

```
                                                ▼
+-------------------------------------------------------------------------------
------+
| MILESTONE 4: VERIFICATION, AUDIT & FINAL WRAP-UP (Phase 4)
|
| - Add mathematical impact calculation modules with zero-division guardrails.
|
| - Connect immutable JSON audit ledger for post-shift supervisor debriefs.
|
| - Execute full test suite (`pytest tests/`) ensuring 100% test pass status.
|
+-------------------------------------------------------------------------------
------+
```

---

13. Transparent Limitations & Hackathon Assumptions

1. **Simulation Sandboxing:** In compliance with safety standards, BusGuard AI operates strictly in a local discrete-event simulation sandbox. No live GTFS-RT dispatch commands are transmitted to active Rapid KL transit assets on road corridors.
2. **Telemetry Frequency Assumptions:** In the absence of high-frequency GPS pings (`$<10\text{ seconds}$`), dead reckoning algorithms assume constant velocity between recorded transit stops.
3. **Standby Fleet Localization:** Standby bus availability is read from simulated depot allocation tables. Standby driver compliance assumes standard statutory Malaysian transport labor rules (maximum 8-hour driving shifts).
4. **Cost Estimations:** Incremental intervention costs (e.g., fuel consumption and driver overtime) utilize standard commercial assumptions (RM 2.20 / km for diesel transit buses) and are explicitly marked as `[SIMULATED COST]` on all interface displays.